Operation Systems

Orange Problem – 4

Name: Rohit s Rajan	**SRN**: PES2UG25CS821	**Section**: K

GitHub: https://github.com/NTXSTEALTH/Orange-4/

Phase 1 |

1 A

```
rohit@StealthPC:~/os-orange$ make test_objects
gcc -o test_objects test_objects.o object.o -lcrypto
rohit@StealthPC:~/os-orange$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aa
fc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
rohit@StealthPC:~/os-orange$
```

1 B

```
rohit@StealthPC:~/os-orange$ find .pes/objects -type f
.pes/objects/96/c0d5c28dd3b719606c36723cd16aefbc73b3dc8b74de61070183357d6968b9
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/c9/6f2fc25394979be592d5cf80c5d78ec792ff13dde17f838449fbcdfbb7df0d
.pes/objects/bc/c7006bbe7a6b6ffa57eec956a8fe3f9ca2fbb1aa1d1e5e7102d5b3db94c380
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/c6/285f25c5d8b32301f6e3b498e7f530f1e4b1e50cece305f983b409cf2a69ef
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/a6/06c7fead004199013e5132fa50894014606bde7b1d46d1581c23954c7f17cb
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
```

Phase 2 |

2 A

```
rohit@StealthPC:~/os-orange$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
rohit@StealthPC:~/os-orange$
```

2 B

```
rohit@StealthPC:~/os-orange$ xxd object.o | head 20
00000000: 7f45 4c46 0201 0100 0000 0000 0000 0000  .ELF............
00000010: 0100 3e00 0100 0000 0000 0000 0000 0000  ..>.............
00000020: 0000 0000 0000 0000 b825 0000 0000 0000  .........%......
00000030: 0000 0000 4000 0000 0000 4000 1000 0f00  ....@.....@.....
00000040: 4157 4156 4155 4154 4989 fc55 5348 81ec  AWAVAUATI..USH..
00000050: 3801 0000 f30f 6f16 660f 6f05 0000 0000  8.....o.f.o.....
00000060: 6448 8b04 2528 0000 0048 8984 2428 0100  dH..%(...H..$(..
00000070: 0031 c0f3 0f6f 6610 f30f 6f5e 2048 8d44  .1...of...o^ H.D
00000080: 2460 660f 6fc8 660f 6ff0 f30f 6f6e 3048  $`f.o.f.o...on0H
00000090: 8d94 2420 0100 0066 0fdb f466 0f71 d408  ..$ ...f...f.q..
rohit@StealthPC:~/os-orange$
```

Phase 3 |

3 A

```
rohit@StealthPC:~/os-orange$ ./pes init
echo "Hello" > hello.txt
./pes add hello.txt
./pes commit -m "Initial commit"

echo "World" >> hello.txt
./pes add hello.txt
./pes commit -m "Add world"

echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"

./pes log
Initialized empty PES repository in .pes/
Committed: a606c7fead00... Initial commit
Committed: bcc7006bbe7a... Add world
Committed: c6285f25c5d8... Add farewell
commit c6285f25c5d8b32301f6e3b498e7f530f1e4b1e50cece305f983b409cf2a69ef
Author: Rohit s Rajan <PES2UG25CS821>
Date:   april 21 2026

    Add farewell

commit bcc7006bbe7a6b6ffa57eec956a8fe3f9ca2fbb1aa1d1e5e7102d5b3db94c380
Author: Rohit s Rajan <PES2UG25CS821>
Date:   april 21 2026

    Add world

commit a606c7fead004199013e5132fa50894014606bde7b1d46d1581c23954c7f17cb
Author: Rohit s Rajan <PES2UG25CS821>
Date:   april 21 2026

    Initial commit

rohit@StealthPC:~/os-orange$
```

3 B

```
rohit@StealthPC:~/os-orange$ cat .pes/index
100644 e67ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc 1776753372 8 bye.txt
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776752549 6 file1.t
xt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776752549 6 file2.t
xt
100644 05c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43 1776753372 12 hello.
txt
rohit@StealthPC:~/os-orange$
```

Phase 4 |

4 A

```
rohit@StealthPC:~/os-orange$ ./pes log
commit c6285f25c5d8b32301f6e3b498e7f530f1e4b1e50cece305f983b409cf2a69ef
Author: Rohit s Rajan <PES2UG25CS821>
Date:   april 21 2026

    Add farewell

commit bcc7006bbe7a6b6ffa57eec956a8fe3f9ca2fbb1aa1d1e5e7102d5b3db94c380
Author: Rohit s Rajan <PES2UG25CS821>
Date:   april 21 2026

    Add world

commit a606c7fead004199013e5132fa50894014606bde7b1d46d1581c23954c7f17cb
Author: Rohit s Rajan <PES2UG25CS821>
Date:   april 21 2026

    Initial commit
```

4 B

```
rohit@StealthPC:~/os-orange$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/96/c0d5c28dd3b719606c36723cd16aefbc73b3dc8b74de61070183357d6968b9
.pes/objects/a6/06c7fead004199013e5132fa50894014606bde7b1d46d1581c23954c7f17cb
.pes/objects/bc/c7006bbe7a6b6ffa57eec956a8fe3f9ca2fbb1aa1d1e5e7102d5b3db94c380
.pes/objects/c6/285f25c5d8b32301f6e3b498e7f530f1e4b1e50cece305f983b409cf2a69ef
.pes/objects/c9/6f2fc25394979be592d5cf80c5d78ec792ff13dde17f838449fbcdfbb7df0d
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
```

4 C

```
rohit@StealthPC:~/os-orange$ cat .pes/refs/heads/main
c6285f25c5d8b32301f6e3b498e7f530f1e4b1e50cece305f983b409cf2a69ef
```

```
rohit@StealthPC:~/os-orange$ cat .pes/HEAD
ref: refs/heads/main
rohit@StealthPC:~/os-orange$
```

Integration Test

```
rohit@StealthPC:~/os-orange$ make test-integration
bash test_sequence.sh
═══ PES-VCS Integration Test ═══

── Repository Initialization ──
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

── Staging Files ──
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


── First Commit ──
Committed: ed2c7324a657... Initial commit

Log after first commit:
commit ed2c7324a657dba1bd4b2a0f9bc4dea01ca10f0a3bcaf8b3eac3d534979134ee
Author: Rohit s Rajan <PES2UG25CS821>
Date:   april 21 2026

    Initial commit


── Second Commit ──
Committed: 7989ea25011d... Update file.txt

── Third Commit ──
Committed: 3295692b1a41... Add farewell

── Full History ──
commit 3295692b1a41249631aa778249d2a47ab4054dd596bf5db7a650d9c8ce4030f8
Author: Rohit s Rajan <PES2UG25CS821>
```

```
    Add farewell

commit 7989ea25011dcef0b77c359daf2d1b489877c022c454f69b0830ac6314b82c7c
Author: Rohit s Rajan <PES2UG25CS821>
Date:   april 21 2026

    Update file.txt

commit ed2c7324a657dba1bd4b2a0f9bc4dea01ca10f0a3bcaf8b3eac3d534979134ee
Author: Rohit s Rajan <PES2UG25CS821>
Date:   april 21 2026

    Initial commit


── Reference Chain ──
HEAD:
ref: refs/heads/main
refs/heads/main:
3295692b1a41249631aa778249d2a47ab4054dd596bf5db7a650d9c8ce4030f8

── Object Store ──
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/32/95692b1a41249631aa778249d2a47ab4054dd596bf5db7a650d9c8ce4030f8
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/79/89ea25011dcef0b77c359daf2d1b489877c022c454f69b0830ac6314b82c7c
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/ed/2c7324a657dba1bd4b2a0f9bc4dea01ca10f0a3bcaf8b3eac3d534979134ee

═══ All integration tests completed ═══
rohit@StealthPC:~/os-orange$
```

Phase 5

| 5 - 1

To implement pes checkout, I'd update .pes/HEAD to point to the new branch, read its commit, and update the working directory to match that tree. It's complex because we have to delete old files, restore new ones, and handle uncommitted changes properly.

| 5 - 2

I'd compare file hashes in the index with actual files. If they don't match, the file is modified. If that file also differs in the target branch, checkout should be refused.

| 5 - 3

In detached HEAD, commits are made but not linked to any branch, so they can get lost. You can recover them using the commit hash and creating a new branch.

| 6 - 1

Start from branch heads, traverse all commits, trees, and blobs, mark reachable ones, and delete the rest. A hash set can be used to track visited objects.

| 6 - 2

GC can delete objects while a commit is still being created, causing broken commits. Git avoids this using locks, careful timing, and by not deleting recent objects immediately.

CODE:

Object.c

```
// object.c — Content-addressable object store
//
// Every piece of data (file contents, directory listings, commits) is stored
// as an "object" named by its SHA-256 hash. Objects are stored under
// .pes/objects/XX/YYYYYY... where XX is the first two hex characters of the
// hash (directory sharding).
//
// PROVIDED functions: compute_hash, object_path, object_exists, hash_to_hex, hex_to_hash
// TODO functions:     object_write, object_read
```


```
// object.c — Content-addressable object store
//
// Every piece of data (file contents, directory listings, commits) is stored
// as an "object" named by its SHA-256 hash. Objects are stored under
// .pes/objects/XX/YYYYYY... where XX is the first two hex characters of the
// hash (directory sharding).
//
// PROVIDED functions: compute_hash, object_path, object_exists, hash_to_hex, hex_to_hash
// TODO functions:     object_write, object_read
```


```
#include "pes.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>

// Self-contained SHA-256 implementation to avoid external OpenSSL dependency.
// Public domain implementation adapted for this repository.

static inline uint32_t rotr(uint32_t x, unsigned int n) {
    return (x >> n) | (x << (32 - n));
}

static void sha256_transform(uint32_t state[8], const uint8_t block[64]) {
    static const uint32_t k[64] = {
        0x428a2f98, 0x71374491, 0xb5c0fbcf, 0xe9b5dba5,
        0x3956c25b, 0x59f111f1, 0x923f82a4, 0xab1c5ed5,
        0xd807aa98, 0x12835b01, 0x243185be, 0x550c7dc3,
        0x72be5d74, 0x80deb1fe, 0x9bdc06a7, 0xc19bf174,
        0xe49b69c1, 0xefbe4786, 0x0fc19dc6, 0x240ca1cc,
        0x2de92c6f, 0x4a7484aa, 0x5cb0a9dc, 0x76f988da,
        0x983e5152, 0xa831c66d, 0xb00327c8, 0xbf597fc7,
        0xc6e00bf3, 0xd5a79147, 0x06ca6351, 0x14292967,
        0x27b70a85, 0x2e1b2138, 0x4d2c6dfc, 0x53380d13,
        0x650a7354, 0x766a0abb, 0x81c2c92e, 0x92722c85,
        0xa2bfe8a1, 0xa81a664b, 0xc24b8b70, 0xc76c51a3,
        0xd192e819, 0xd6990624, 0xf40e3585, 0x106aa070,
        0x19a4c116, 0x1e376c08, 0x2748774c, 0x34b0bcb5,
        0x391c0cb3, 0x4ed8aa4a, 0x5b9cca4f, 0x682e6ff3,
        0x748f82ee, 0x78a5636f, 0x84c87814, 0x8cc70208,
        0x90befffa, 0xa4506ceb, 0xbef9a3f7, 0xc67178f2
    };

    uint32_t w[64];
    for (int i = 0; i < 16; i++) {
        w[i] = ((uint32_t)block[i * 4] << 24) |
               ((uint32_t)block[i * 4 + 1] << 16) |
               ((uint32_t)block[i * 4 + 2] << 8) |
```

```
        ((uint32_t)block[i * 4 + 3]);
}
for (int i = 16; i < 64; i++) {
    uint32_t s0 = rotr(w[i - 15], 7) ^ rotr(w[i - 15], 18) ^ (w[i - 15] >> 3);
    uint32_t s1 = rotr(w[i - 2], 17) ^ rotr(w[i - 2], 19) ^ (w[i - 2] >> 10);
    w[i] = w[i - 16] + s0 + w[i - 7] + s1;
}

uint32_t a = state[0];
uint32_t b = state[1];
uint32_t c = state[2];
uint32_t d = state[3];
uint32_t e = state[4];
uint32_t f = state[5];
uint32_t g = state[6];
uint32_t h = state[7];

for (int i = 0; i < 64; i++) {
    uint32_t S1 = rotr(e, 6) ^ rotr(e, 11) ^ rotr(e, 25);
    uint32_t ch = (e & f) ^ (~e & g);
    uint32_t temp1 = h + S1 + ch + k[i] + w[i];
    uint32_t S0 = rotr(a, 2) ^ rotr(a, 13) ^ rotr(a, 22);
    uint32_t maj = (a & b) ^ (a & c) ^ (b & c);
    uint32_t temp2 = S0 + maj;

    h = g;
    g = f;
    f = e;
    e = d + temp1;
    d = c;
    c = b;
    b = a;
    a = temp1 + temp2;
}

state[0] += a;
```

```
    state[1] += b;
    state[2] += c;
    state[3] += d;
    state[4] += e;
    state[5] += f;
    state[6] += g;
    state[7] += h;
}

void compute_hash(const void *data, size_t len, ObjectID *id_out) {
    uint64_t bit_len = (uint64_t)len * 8;
    uint8_t buffer[64];
    uint32_t state[8] = {
        0x6a09e667, 0xbb67ae85, 0x3c6ef372, 0xa54ff53a,
        0x510e527f, 0x9b05688c, 0x1f83d9ab, 0x5be0cd19
    };

    size_t processed = 0;
    while (len - processed >= 64) {
        sha256_transform(state, (const uint8_t *)data + processed);
        processed += 64;
    }

    size_t remaining = len - processed;
    memcpy(buffer, (const uint8_t *)data + processed, remaining);
    buffer[remaining] = 0x80;
    size_t pad_len = (remaining < 56) ? (56 - remaining) : (120 - remaining);
    memset(buffer + remaining + 1, 0, pad_len);

    buffer[56] = (uint8_t)(bit_len >> 56);
    buffer[57] = (uint8_t)(bit_len >> 48);
    buffer[58] = (uint8_t)(bit_len >> 40);
    buffer[59] = (uint8_t)(bit_len >> 32);
    buffer[60] = (uint8_t)(bit_len >> 24);
    buffer[61] = (uint8_t)(bit_len >> 16);
    buffer[62] = (uint8_t)(bit_len >> 8);
```

```
    buffer[63] = (uint8_t)(bit_len);

    sha256_transform(state, buffer);
    if (remaining + 1 + pad_len > 64) {
        // Process second block when padding didn't fit in first block
        uint8_t second_block[64];
        memset(second_block, 0, sizeof(second_block));
        second_block[56] = (uint8_t)(bit_len >> 56);
        second_block[57] = (uint8_t)(bit_len >> 48);
        second_block[58] = (uint8_t)(bit_len >> 40);
        second_block[59] = (uint8_t)(bit_len >> 32);
        second_block[60] = (uint8_t)(bit_len >> 24);
        second_block[61] = (uint8_t)(bit_len >> 16);
        second_block[62] = (uint8_t)(bit_len >> 8);
        second_block[63] = (uint8_t)(bit_len);
        sha256_transform(state, second_block);
    }

    for (int i = 0; i < 8; i++) {
        id_out->hash[i * 4]     = (uint8_t)(state[i] >> 24);
        id_out->hash[i * 4 + 1] = (uint8_t)(state[i] >> 16);
        id_out->hash[i * 4 + 2] = (uint8_t)(state[i] >> 8);
        id_out->hash[i * 4 + 3] = (uint8_t)(state[i]);
    }
}

// ——— PROVIDED ———————————————————————————————————————————————————————————————————————————————

void hash_to_hex(const ObjectID *id, char *hex_out) {
    for (int i = 0; i < HASH_SIZE; i++) {
        sprintf(hex_out + i * 2, "%02x", id->hash[i]);
    }
    hex_out[HASH_HEX_SIZE] = '\0';
}
```

```
int hex_to_hash(const char *hex, ObjectID *id_out) {
    if (strlen(hex) < HASH_HEX_SIZE) return -1;
    for (int i = 0; i < HASH_SIZE; i++) {
        unsigned int byte;
        if (sscanf(hex + i * 2, "%2x", &byte) != 1) return -1;
        id_out->hash[i] = (uint8_t)byte;
    }
    return 0;
}

// Get the filesystem path where an object should be stored.
// Format: .pes/objects/XX/YYYYYYYY...
// The first 2 hex chars form the shard directory; the rest is the filename.
void object_path(const ObjectID *id, char *path_out, size_t path_size) {
    char hex[HASH_HEX_SIZE + 1];
    hash_to_hex(id, hex);
    snprintf(path_out, path_size, "%s/%.2s/%s", OBJECTS_DIR, hex, hex + 2);
}

int object_exists(const ObjectID *id) {
    char path[512];
    object_path(id, path, sizeof(path));
    return access(path, F_OK) == 0;
}

// ——— TODO: Implement these ———————————————————————————————————————————————

// Write an object to the store.
//
// Object format on disk:
//   "<type> <size>\0<data>"
//   where <type> is "blob", "tree", or "commit"
//   and <size> is the decimal string of the data length
//
// Steps:
//   1. Build the full object: header ("blob 16\0") + data
```

```
//   2. Compute SHA-256 hash of the FULL object (header + data)
//   3. Check if object already exists (deduplication) — if so, just return success
//   4. Create shard directory (.pes/objects/XX/) if it doesn't exist
//   5. Write to a temporary file in the same shard directory
//   6. fsync() the temporary file to ensure data reaches disk
//   7. rename() the temp file to the final path (atomic on POSIX)
//   8. Open and fsync() the shard directory to persist the rename
//   9. Store the computed hash in *id_out

// HINTS - Useful syscalls and functions for this phase:
//   - sprintf / snprintf : formatting the header string
//   - compute_hash       : hashing the combined header + data
//   - object_exists      : checking for deduplication
//   - mkdir              : creating the shard directory (use mode 0755)
//   - open, write, close : creating and writing to the temp file
//                          (Use O_CREAT | O_WRONLY | O_TRUNC, mode 0644)
//   - fsync              : flushing the file descriptor to disk
//   - rename             : atomically moving the temp file to the final path
//

//
// Returns 0 on success, -1 on error.
int object_write(ObjectType type, const void *data, size_t len, ObjectID *id_out) {
    // Step 1: Build the full object: header ("blob 16\0") + data
    const char *type_str;
    switch (type) {
        case OBJ_BLOB:   type_str = "blob"; break;
        case OBJ_TREE:   type_str = "tree"; break;
        case OBJ_COMMIT: type_str = "commit"; break;
        default: return -1;
    }

    // Build header string
    char header[256];
    snprintf(header, sizeof(header), "%s %lu", type_str, (unsigned long)len);
    size_t header_len = strlen(header);
```

```
// Create full object: header + '\0' + data
size_t full_len = header_len + 1 + len;
void *full_obj = malloc(full_len);
if (!full_obj) return -1;

memcpy(full_obj, header, header_len);
((char *)full_obj)[header_len] = '\0';
if (len > 0) memcpy((char *)full_obj + header_len + 1, data, len);

// Step 2: Compute SHA-256 hash of the FULL object
compute_hash(full_obj, full_len, id_out);

// Step 3: Check if object already exists (deduplication)
if (object_exists(id_out)) {
    free(full_obj);
    return 0;
}

// Step 4: Create shard directory if it doesn't exist
char shard_dir[512];
char hex[HASH_HEX_SIZE + 1];
hash_to_hex(id_out, hex);
snprintf(shard_dir, sizeof(shard_dir), "%s/%.2s", OBJECTS_DIR, hex);
mkdir(shard_dir, 0755);

// Step 5: Write to a temporary file
char path[512];
object_path(id_out, path, sizeof(path));

char tmp_path[520];
if (snprintf(tmp_path, sizeof(tmp_path), "%s.tmp", path) >= (int)sizeof(tmp_path)) {
    free(full_obj);
    return -1;
}
```

```
int fd = open(tmp_path, O_CREAT | O_WRONLY | O_TRUNC, 0644);
if (fd < 0) {
    free(full_obj);
    return -1;
}

if (write(fd, full_obj, full_len) != (ssize_t)full_len) {
    close(fd);
    unlink(tmp_path);
    free(full_obj);
    return -1;
}

// Step 6: fsync() the temporary file to ensure data reaches disk
if (fsync(fd) != 0) {
    close(fd);
    unlink(tmp_path);
    free(full_obj);
    return -1;
}
close(fd);

// Step 7: rename() the temp file to the final path (atomic on POSIX)
if (rename(tmp_path, path) != 0) {
    free(full_obj);
    return -1;
}

// Step 8: fsync() the shard directory to persist the rename
fd = open(shard_dir, O_RDONLY);
if (fd >= 0) {
    fsync(fd);
    close(fd);
}

free(full_obj);
```

```
    return 0;
}

// Read an object from the store.
//
// Steps:
//   1. Build the file path from the hash using object_path()
//   2. Open and read the entire file
//   3. Parse the header to extract the type string and size
//   4. Verify integrity: recompute the SHA-256 of the file contents
//      and compare to the expected hash (from *id). Return -1 if mismatch.
//   5. Set *type_out to the parsed ObjectType
//   6. Allocate a buffer, copy the data portion (after the \0), set *data_out and *len_out
//
// HINTS - Useful syscalls and functions for this phase:
//   - object_path        : getting the target file path
//   - fopen, fread, fseek: reading the file into memory
//   - memchr             : safely finding the '\0' separating header and data
//   - strncmp            : parsing the type string ("blob", "tree", "commit")
//   - compute_hash       : re-hashing the read data for integrity verification
//   - memcmp             : comparing the computed hash against the requested hash
//   - malloc, memcpy     : allocating and returning the extracted data
//
// The caller is responsible for calling free(*data_out).
// Returns 0 on success, -1 on error (file not found, corrupt, etc.).
int object_read(const ObjectID *id, ObjectType *type_out, void **data_out, size_t *len_out) {
    char path[512];
    object_path(id, path, sizeof(path));

    // Step 1: Open and read the entire file
    FILE *f = fopen(path, "rb");
    if (!f) return -1;

    // Read file into memory
    fseek(f, 0, SEEK_END);
    long file_size = ftell(f);
```

```
if (file_size < 0) {
    fclose(f);
    return -1;
}
fseek(f, 0, SEEK_SET);

void *file_data = malloc(file_size);
if (!file_data) {
    fclose(f);
    return -1;
}

if (fread(file_data, 1, file_size, f) != (size_t)file_size) {
    free(file_data);
    fclose(f);
    return -1;
}
fclose(f);

// Step 2 & 3: Parse the header and verify integrity
// Find the null terminator that separates header and data
void *null_ptr = memchr(file_data, '\0', file_size);
if (!null_ptr) {
    free(file_data);
    return -1;
}

size_t header_len = (char *)null_ptr - (char *)file_data;
char header_str[256];
if (header_len >= sizeof(header_str)) {
    free(file_data);
    return -1;
}
memcpy(header_str, file_data, header_len);
header_str[header_len] = '\0';
```

```
// Parse type and size from header
char type_name[32];
size_t parsed_size;
if (sscanf(header_str, "%31s %zu", type_name, &parsed_size) != 2) {
    free(file_data);
    return -1;
}

// Map type string to ObjectType
ObjectType type;
if (strcmp(type_name, "blob") == 0)       type = OBJ_BLOB;
else if (strcmp(type_name, "tree") == 0)  type = OBJ_TREE;
else if (strcmp(type_name, "commit") == 0) type = OBJ_COMMIT;
else {
    free(file_data);
    return -1;
}

// Verify that the parsed size matches actual data
size_t data_len = file_size - header_len - 1;
if (data_len != parsed_size) {
    free(file_data);
    return -1;
}

// Verify integrity: recompute SHA-256 and compare
ObjectID computed_id;
compute_hash(file_data, file_size, &computed_id);
if (memcmp(&computed_id, id, sizeof(ObjectID)) != 0) {
    free(file_data);
    return -1;  // Hash mismatch - corruption detected
}

// Step 5 & 6: Extract data portion and return it
void *data_out_buf = malloc(data_len + 1);
if (!data_out_buf) {
```

```
        free(file_data);
        return -1;
    }
    if (data_len > 0) {
        memcpy(data_out_buf, (char *)file_data + header_len + 1, data_len);
    }
    ((char *)data_out_buf)[data_len] = '\0';

    *type_out = type;
    *data_out = data_out_buf;
    *len_out = data_len;

    free(file_data);
    return 0;
}
```

Test Object.c

```c
// test_objects.c — Phase 1 test program
//
// Compile and run:
//   gcc -Wall -Wextra -O2 -o test_objects test_objects.c object.c -lcrypto
//   ./test_objects

#include "pes.h"
#include <stdio.h>
#include <string.h>
#include <assert.h>
#include <stdlib.h>

// Forward declarations for object.c functions
int object_write(ObjectType type, const void *data, size_t len, ObjectID *id_out);
int object_read(const ObjectID *id, ObjectType *type_out, void **data_out, size_t *len_out);
int object_exists(const ObjectID *id);
void object_path(const ObjectID *id, char *path_out, size_t path_size);

void test_blob_storage(void) {
    const char *content = "Hello, PES-VCS!\n";
    ObjectID id;

    int rc = object_write(OBJ_BLOB, content, strlen(content), &id);
    assert(rc == 0);

    char hex[HASH_HEX_SIZE + 1];
    hash_to_hex(&id, hex);
    printf("Stored blob with hash: %s\n", hex);

    char path[512];
```

```
    object_path(&id, path, sizeof(path));
    printf("Object stored at: %s\n", path);

    // Read it back and verify
    ObjectType type;
    void *data;
    size_t len;
    rc = object_read(&id, &type, &data, &len);
    assert(rc == 0);
    assert(type == OBJ_BLOB);
    assert(len == strlen(content));
    assert(memcmp(data, content, len) == 0);
    free(data);

    printf("PASS: blob storage\n");
}

void test_deduplication(void) {
    const char *content = "Duplicate content\n";
    ObjectID id1, id2;

    object_write(OBJ_BLOB, content, strlen(content), &id1);
    object_write(OBJ_BLOB, content, strlen(content), &id2);

    assert(memcmp(&id1, &id2, sizeof(ObjectID)) == 0);

    printf("PASS: deduplication\n");
}

void test_integrity(void) {
    const char *content = "Test integrity\n";
    ObjectID id;
```

```
    object_write(OBJ_BLOB, content, strlen(content), &id);

    // Corrupt the stored file
    char path[512];
    object_path(&id, path, sizeof(path));

    FILE *f = fopen(path, "r+b");
    assert(f != NULL);
    fseek(f, 20, SEEK_SET);
    fputc('X', f);
    fclose(f);

    // Read should detect corruption
    ObjectType type;
    void *data;
    size_t len;
    int rc = object_read(&id, &type, &data, &len);
    assert(rc == -1);  // Must fail integrity check

    printf("PASS: integrity check\n");
}

int main(void) {
    // Clean slate
    int rc __attribute__((unused));
    rc = system("rm -rf .pes");
    rc = system("mkdir -p .pes/objects .pes/refs/heads");

    test_blob_storage();
    test_deduplication();
    test_integrity();
```

```
    printf("\nAll Phase 1 tests passed.\n");
    return 0;
}
```

Commit.c

```
// commit.c — Commit creation and history traversal
//
// Commit object format (stored as text, one field per line):
//
//   tree <64-char-hex-hash>
//   parent <64-char-hex-hash>        ← omitted for the first commit
//   author <name> <unix-timestamp>
//   committer <name> <unix-timestamp>
//
//   <commit message>
//
// Note: there is a blank line between the headers and the message.
//
// PROVIDED functions: commit_parse, commit_serialize, commit_walk, head_read, head_update
// TODO functions:     commit_create

#include "commit.h"
#include "index.h"
#include "tree.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <inttypes.h>
#include <time.h>
#include <unistd.h>
#include <fcntl.h>

// Forward declarations (implemented in object.c)
```

```
int object_write(ObjectType type, const void *data, size_t len, ObjectID *id_out);
int object_read(const ObjectID *id, ObjectType *type_out, void **data_out, size_t *len_out);

// —— PROVIDED ————————————————————————————————————————————————————————————————————————————————

// Parse raw commit data into a Commit struct.
int commit_parse(const void *data, size_t len, Commit *commit_out) {
    (void)len;
    const char *p = (const char *)data;
    char hex[HASH_HEX_SIZE + 1];

    // "tree <hex>\n"
    if (sscanf(p, "tree %64s\n", hex) != 1) return -1;
    if (hex_to_hash(hex, &commit_out->tree) != 0) return -1;
    p = strchr(p, '\n') + 1;

    // optional "parent <hex>\n"
    if (strncmp(p, "parent ", 7) == 0) {
        if (sscanf(p, "parent %64s\n", hex) != 1) return -1;
        if (hex_to_hash(hex, &commit_out->parent) != 0) return -1;
        commit_out->has_parent = 1;
        p = strchr(p, '\n') + 1;
    } else {
        commit_out->has_parent = 0;
    }

    // "author <name> <timestamp>\n"
    char author_buf[256];
    uint64_t ts;
    if (sscanf(p, "author %255[^\n]\n", author_buf) != 1) return -1;
    // split off trailing timestamp
    char *last_space = strrchr(author_buf, ' ');
    if (!last_space) return -1;
    ts = (uint64_t)strtoull(last_space + 1, NULL, 10);
    *last_space = '\0';
```

```
    snprintf(commit_out->author, sizeof(commit_out->author), "%s", author_buf);
    commit_out->timestamp = ts;
    p = strchr(p, '\n') + 1;  // skip author line
    p = strchr(p, '\n') + 1;  // skip committer line
    p = strchr(p, '\n') + 1;  // skip blank line

    snprintf(commit_out->message, sizeof(commit_out->message), "%s", p);
    return 0;
}

// Serialize a Commit struct to the text format.
// Caller must free(*data_out).
int commit_serialize(const Commit *commit, void **data_out, size_t *len_out) {
    char tree_hex[HASH_HEX_SIZE + 1];
    char parent_hex[HASH_HEX_SIZE + 1];
    hash_to_hex(&commit->tree, tree_hex);

    char buf[8192];
    int n = 0;
    n += snprintf(buf + n, sizeof(buf) - n, "tree %s\n", tree_hex);
    if (commit->has_parent) {
        hash_to_hex(&commit->parent, parent_hex);
        n += snprintf(buf + n, sizeof(buf) - n, "parent %s\n", parent_hex);
    }
    n += snprintf(buf + n, sizeof(buf) - n,
                  "author %s %" PRIu64 "\n"
                  "committer %s %" PRIu64 "\n"
                  "\n"
                  "%s",
                  commit->author, commit->timestamp,
                  commit->author, commit->timestamp,
                  commit->message);

    *data_out = malloc(n + 1);
    if (!*data_out) return -1;
    memcpy(*data_out, buf, n + 1);
```

```
    *len_out = (size_t)n;
    return 0;
}

// Walk commit history from HEAD to the root.
int commit_walk(commit_walk_fn callback, void *ctx) {
    ObjectID id;
    if (head_read(&id) != 0) return -1;

    while (1) {
        ObjectType type;
        void *raw;
        size_t raw_len;
        if (object_read(&id, &type, &raw, &raw_len) != 0) return -1;

        Commit c;
        int rc = commit_parse(raw, raw_len, &c);
        free(raw);
        if (rc != 0) return -1;

        callback(&id, &c, ctx);

        if (!c.has_parent) break;
        id = c.parent;
    }
    return 0;
}

// Read the current HEAD commit hash.
int head_read(ObjectID *id_out) {
    FILE *f = fopen(HEAD_FILE, "r");
    if (!f) return -1;
    char line[512];
    if (!fgets(line, sizeof(line), f)) { fclose(f); return -1; }
    fclose(f);
    line[strcspn(line, "\r\n")] = '\0'; // strip newline
```

```
    char ref_path[512];
    if (strncmp(line, "ref: ", 5) == 0) {
        snprintf(ref_path, sizeof(ref_path), "%s/%s", PES_DIR, line + 5);
        f = fopen(ref_path, "r");
        if (!f) return -1; // Branch exists but has no commits yet
        if (!fgets(line, sizeof(line), f)) { fclose(f); return -1; }
        fclose(f);
        line[strcspn(line, "\r\n")] = '\0';
    }
    return hex_to_hash(line, id_out);
}

// Update the current branch ref to point to a new commit atomically.
int head_update(const ObjectID *new_commit) {
    FILE *f = fopen(HEAD_FILE, "r");
    if (!f) return -1;
    char line[512];
    if (!fgets(line, sizeof(line), f)) { fclose(f); return -1; }
    fclose(f);
    line[strcspn(line, "\r\n")] = '\0';

    char target_path[520];
    if (strncmp(line, "ref: ", 5) == 0) {
        snprintf(target_path, sizeof(target_path), "%s/%s", PES_DIR, line + 5);
    } else {
        snprintf(target_path, sizeof(target_path), "%s", HEAD_FILE); // Detached HEAD
    }

    char tmp_path[528];
    snprintf(tmp_path, sizeof(tmp_path), "%s.tmp", target_path);

    f = fopen(tmp_path, "w");
    if (!f) return -1;

    char hex[HASH_HEX_SIZE + 1];
```

```
    hash_to_hex(new_commit, hex);
    fprintf(f, "%s\n", hex);

    fflush(f);
    fsync(fileno(f));
    fclose(f);

    return rename(tmp_path, target_path);
}

// ─── TODO: Implement these ──────────────────────────────────────────────────────────────────────

// Helper to get the author string from PES_AUTHOR environment variable
static void get_author(char *author_buf, size_t size) {
    const char *env_author = getenv("PES_AUTHOR");
    if (env_author) {
        snprintf(author_buf, size, "%s", env_author);
    } else {
        snprintf(author_buf, size, "HARISH <PES2UG24AM229@pes.edu>");
    }
}

// Create a new commit from the current staging area.
int commit_create(const char *message, ObjectID *commit_id_out) {
    // Step 1: Build a tree from the index
    ObjectID tree_id;
    if (tree_from_index(&tree_id) != 0) {
        return -1;
    }

    // Step 2: Get the parent commit hash (if any)
    ObjectID parent_id;
    int has_parent = 0;
    if (head_read(&parent_id) == 0) {
        has_parent = 1;
    }
```

```
    // Step 3: Get author information and timestamp
    Commit commit = {0};
    commit.tree = tree_id;
    commit.has_parent = has_parent;
    if (has_parent) {
        commit.parent = parent_id;
    }
    get_author(commit.author, sizeof(commit.author));
    commit.timestamp = time(NULL);
    snprintf(commit.message, sizeof(commit.message), "%s", message);

    // Step 4: Serialize the commit
    void *commit_data;
    size_t commit_len;
    if (commit_serialize(&commit, &commit_data, &commit_len) != 0) {
        return -1;
    }

    // Step 5: Write the commit object
    if (object_write(OBJ_COMMIT, commit_data, commit_len, commit_id_out) != 0) {
        free(commit_data);
        return -1;
    }

    // Step 6: Update HEAD to point to the new commit
    if (head_update(commit_id_out) != 0) {
        free(commit_data);
        return -1;
    }

    free(commit_data);
    return 0;
}
```

Index.c

```
// index.c — Staging area implementation
//
// Text format of .pes/index (one entry per line, sorted by path):
//
//   <mode-octal> <64-char-hex-hash> <mtime-seconds> <size> <path>
//
// Example:
//   100644 a1b2c3d4e5f6...  1699900000 42 README.md
//   100644 f7e8d9c0b1a2...  1699900100 128 src/main.c
//
// This is intentionally a simple text format. No magic numbers, no
// binary parsing. The focus is on the staging area CONCEPT (tracking
// what will go into the next commit) and ATOMIC WRITES (temp+rename).
//
// PROVIDED functions: index_find, index_remove, index_status
// TODO functions:     index_load, index_save, index_add



#include "index.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <dirent.h>
#include <inttypes.h>

// Forward declaration for object_write from object.c
int object_write(ObjectType type, const void *data, size_t len, ObjectID *id_out);

// ——— PROVIDED ———————————————
```

```
IndexEntry* index_find(Index *index, const char *path) {
    for (int i = 0; i < index->count; i++) {
        if (strcmp(index->entries[i].path, path) == 0)
            return &index->entries[i];
    }
    return NULL;
}

// Remove a file from the index.
// Returns 0 on success, -1 if path not in index.
int index_remove(Index *index, const char *path) {
    for (int i = 0; i < index->count; i++) {
        if (strcmp(index->entries[i].path, path) == 0) {
            int remaining = index->count - i - 1;
            if (remaining > 0)
                memmove(&index->entries[i], &index->entries[i + 1],
                        remaining * sizeof(IndexEntry));
            index->count--;
            return index_save(index);
        }
    }
    fprintf(stderr, "error: '%s' is not in the index\n", path);
    return -1;
}

// Print the status of the working directory.
//
// Identifies files that are staged, unstaged (modified/deleted in working dir),
// and untracked (present in working dir but not in index).
// Returns 0.
int index_status(const Index *index) {
    printf("Staged changes:\n");
    int staged_count = 0;
    // Note: A true Git implementation deeply diffs against the HEAD tree here.
    // For this lab, displaying indexed files represents the staging intent.
    for (int i = 0; i < index->count; i++) {
```

```
    printf("  staged:     %s\n", index->entries[i].path);
    staged_count++;
}
if (staged_count == 0) printf("  (nothing to show)\n");
printf("\n");

printf("Unstaged changes:\n");
int unstaged_count = 0;
for (int i = 0; i < index->count; i++) {
    struct stat st;
    if (stat(index->entries[i].path, &st) != 0) {
        printf("  deleted:    %s\n", index->entries[i].path);
        unstaged_count++;
    } else {
        // Fast diff: check metadata instead of re-hashing file content
        if (st.st_mtime != (time_t)index->entries[i].mtime_sec || st.st_size != (off_t)index->entries[i].size) {
            printf("  modified:   %s\n", index->entries[i].path);
            unstaged_count++;
        }
    }
}
if (unstaged_count == 0) printf("  (nothing to show)\n");
printf("\n");

printf("Untracked files:\n");
int untracked_count = 0;
DIR *dir = opendir(".");
if (dir) {
    struct dirent *ent;
    while ((ent = readdir(dir)) != NULL) {
        // Skip hidden directories, parent directories, and build artifacts
        if (strcmp(ent->d_name, ".") == 0 || strcmp(ent->d_name, "..") == 0) continue;
        if (strcmp(ent->d_name, ".pes") == 0) continue;
        if (strcmp(ent->d_name, "pes") == 0) continue; // compiled executable
        if (strstr(ent->d_name, ".o") != NULL) continue; // object files
```

```
            // Check if file is tracked in the index
            int is_tracked = 0;
            for (int i = 0; i < index->count; i++) {
                if (strcmp(index->entries[i].path, ent->d_name) == 0) {
                    is_tracked = 1;
                    break;
                }
            }

            if (!is_tracked) {
                struct stat st;
                stat(ent->d_name, &st);
                if (S_ISREG(st.st_mode)) { // Only list regular files for simplicity
                    printf("  untracked:  %s\n", ent->d_name);
                    untracked_count++;
                }
            }
        }
        closedir(dir);
    }
    if (untracked_count == 0) printf("  (nothing to show)\n");
    printf("\n");

    return 0;
}

// ——— TODO: Implement these ———————————————————————————————————————————————

// Load the index from .pes/index.
//
// HINTS - Useful functions:
//   - fopen (with "r"), fscanf, fclose : reading the text file line by line
//   - hex_to_hash                      : converting the parsed string to ObjectID
//
// Returns 0 on success, -1 on error.
int index_load(Index *index) {
```

```
    // Initialize the entire struct to zero
    memset(index, 0, sizeof(Index));

    FILE *f = fopen(INDEX_FILE, "r");
    if (!f) {
        // File doesn't exist yet (no files staged) - return empty index
        return 0;
    }

    char line[1024];
    while (fgets(line, sizeof(line), f) && index->count < MAX_INDEX_ENTRIES) {
        IndexEntry *entry = &index->entries[index->count];

        // Parse: <mode-octal> <64-char-hex-hash> <mtime-seconds> <size> <path>
        char hash_hex[HASH_HEX_SIZE + 1];
        uint64_t mtime;
        uint32_t size;
        char path[512];

        int parsed = sscanf(line, "%o %64s %" PRIu64 " %" PRIu32 " %511s",
                            &entry->mode, hash_hex, &mtime, &size, path);
        if (parsed != 5) continue;

        entry->mtime_sec = mtime;
        entry->size = size;
        snprintf(entry->path, sizeof(entry->path), "%s", path);

        if (hex_to_hash(hash_hex, &entry->hash) != 0) continue;

        index->count++;
    }
    fclose(f);
    return 0;
}

// Save the index to .pes/index atomically.
```

```
//
// HINTS - Useful functions and syscalls:
//   - qsort                          : sorting the entries array by path
//   - fopen (with "w"), fprintf      : writing to the temporary file
//   - hash_to_hex                    : converting ObjectID for text output
//   - fflush, fileno, fsync, fclose  : flushing userspace buffers and syncing to disk
//   - rename                         : atomically moving the temp file over the old index
//
// Returns 0 on success, -1 on error.
int index_save(const Index *index) {
    // Create an array of pointers to sort entries without copying the entire struct
    // (the Index struct is too large to copy on the stack)
    IndexEntry **entry_ptrs = malloc(index->count * sizeof(IndexEntry *));
    if (!entry_ptrs) return -1;

    for (int i = 0; i < index->count; i++) {
        entry_ptrs[i] = (IndexEntry *)&index->entries[i];
    }

    // Helper for qsort to compare pointers to entries by path
    int compare_entry_ptrs(const void *a, const void *b) {
        return strcmp((*((const IndexEntry **)a))->path, (*((const IndexEntry **)b))->path);
    }

    // Sort the pointers
    qsort(entry_ptrs, index->count, sizeof(IndexEntry *), compare_entry_ptrs);

    // Write to temporary file
    char tmp_path[512 + 4];
    snprintf(tmp_path, sizeof(tmp_path), "%s.tmp", INDEX_FILE);

    FILE *f = fopen(tmp_path, "w");
    if (!f) {
        free(entry_ptrs);
        return -1;
    }
```

```
    for (int i = 0; i < index->count; i++) {
        const IndexEntry *entry = entry_ptrs[i];
        char hash_hex[HASH_HEX_SIZE + 1];
        hash_to_hex(&entry->hash, hash_hex);

        fprintf(f, "%o %s %" PRIu64 " %" PRIu32 " %s\n",
                entry->mode, hash_hex, entry->mtime_sec, entry->size, entry->path);
    }

    // Flush buffers to disk
    fflush(f);
    fsync(fileno(f));
    fclose(f);

    // Atomically move temp file to final location
    int result = rename(tmp_path, INDEX_FILE);
    free(entry_ptrs);
    return result;
}

// Stage a file for the next commit.
//
// HINTS - Useful functions and syscalls:
//   - fopen, fread, fclose            : reading the target file's contents
//   - object_write                    : saving the contents as OBJ_BLOB
//   - stat / lstat                    : getting file metadata (size, mtime, mode)
//   - index_find                      : checking if the file is already staged
//
// Returns 0 on success, -1 on error.
int index_add(Index *index, const char *path) {
    // Read the file contents
    FILE *f = fopen(path, "rb");
    if (!f) {
        fprintf(stderr, "error: failed to open '%s'\n", path);
        return -1;
```

```
}

fseek(f, 0, SEEK_END);
long file_size = ftell(f);
fseek(f, 0, SEEK_SET);

if (file_size < 0) {
    fclose(f);
    return -1;
}

void *file_data = NULL;
if (file_size > 0) {
    file_data = malloc(file_size);
    if (!file_data) {
        fclose(f);
        return -1;
    }

    if (fread(file_data, 1, file_size, f) != (size_t)file_size) {
        free(file_data);
        fclose(f);
        return -1;
    }
}
fclose(f);

// Write as a blob object
ObjectID blob_id;
if (object_write(OBJ_BLOB, file_data, (size_t)file_size, &blob_id) != 0) {
    if (file_data) free(file_data);
    return -1;
}

// Get file metadata
struct stat st;
```

```
    if (stat(path, &st) != 0) {
        if (file_data) free(file_data);
        return -1;
    }

    // Find or create index entry for this path
    IndexEntry *entry = index_find(index, path);
    if (!entry) {
        if (index->count >= MAX_INDEX_ENTRIES) {
            if (file_data) free(file_data);
            return -1;
        }
        entry = &index->entries[index->count];
        index->count++;
    }

    // Update the entry
    entry->mode = (st.st_mode & S_IXUSR) ? 0100755 : 0100644;
    entry->hash = blob_id;
    entry->mtime_sec = st.st_mtime;
    entry->size = st.st_size;
    strncpy(entry->path, path, sizeof(entry->path) - 1);
    entry->path[sizeof(entry->path) - 1] = '\0';

    if (file_data) free(file_data);

    // Save the updated index
    return index_save(index);
}
```

TREE.c

```
// tree.c — Tree object serialization and construction
//
// PROVIDED functions: get_file_mode, tree_parse, tree_serialize
// TODO functions:     tree_from_index
//
// Binary tree format (per entry, concatenated with no separators):
//   "<mode-as-ascii-octal> <name>\0<32-byte-binary-hash>"
//
// Example single entry (conceptual):
//   "100644 hello.txt\0" followed by 32 raw bytes of SHA-256

#include "tree.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <dirent.h>
#include <sys/stat.h>

#include "index.h"
#include "pes.h"

int object_write(ObjectType type, const void *data, size_t len, ObjectID *id_out);


// ——— Mode Constants ———————————————————————————————————————————————————————

#define MODE_FILE      0100644
#define MODE_EXEC      0100755
#define MODE_DIR       0040000

// ——— PROVIDED ————————————————————————————————————————————————————————————


// Determine the object mode for a filesystem path.
uint32_t get_file_mode(const char *path) {
```

```
    struct stat st;
    if (lstat(path, &st) != 0) return 0;

    if (S_ISDIR(st.st_mode))  return MODE_DIR;
    if (st.st_mode & S_IXUSR) return MODE_EXEC;
    return MODE_FILE;
}

// Parse binary tree data into a Tree struct safely.
// Returns 0 on success, -1 on parse error.
int tree_parse(const void *data, size_t len, Tree *tree_out) {
    tree_out->count = 0;
    const uint8_t *ptr = (const uint8_t *)data;
    const uint8_t *end = ptr + len;

    while (ptr < end && tree_out->count < MAX_TREE_ENTRIES) {
        TreeEntry *entry = &tree_out->entries[tree_out->count];

        const uint8_t *space = memchr(ptr, ' ', end - ptr);
        if (!space) return -1;

        char mode_str[16] = {0};
        size_t mode_len = space - ptr;
        if (mode_len >= sizeof(mode_str)) return -1;
        memcpy(mode_str, ptr, mode_len);
        entry->mode = strtol(mode_str, NULL, 8);

        ptr = space + 1;

        const uint8_t *null_byte = memchr(ptr, '\0', end - ptr);
        if (!null_byte) return -1;

        size_t name_len = null_byte - ptr;
        if (name_len >= sizeof(entry->name)) return -1;
        memcpy(entry->name, ptr, name_len);
        entry->name[name_len] = '\0';
```

```
        ptr = null_byte + 1;

        if (ptr + HASH_SIZE > end) return -1;
        memcpy(entry->hash.hash, ptr, HASH_SIZE);
        ptr += HASH_SIZE;

        tree_out->count++;
    }
    return 0;
}

// Helper for qsort
static int compare_tree_entries(const void *a, const void *b) {
    return strcmp(((const TreeEntry *)a)->name, ((const TreeEntry *)b)->name);
}

// Serialize tree
int tree_serialize(const Tree *tree, void **data_out, size_t *len_out) {
    size_t max_size = tree->count * 296;
    uint8_t *buffer = malloc(max_size);
    if (!buffer) return -1;

    Tree sorted_tree = *tree;
    qsort(sorted_tree.entries, sorted_tree.count, sizeof(TreeEntry), compare_tree_entries);

    size_t offset = 0;
    for (int i = 0; i < sorted_tree.count; i++) {
        const TreeEntry *entry = &sorted_tree.entries[i];

        int written = sprintf((char *)buffer + offset, "%o %s", entry->mode, entry->name);
        offset += written + 1;

        memcpy(buffer + offset, entry->hash.hash, HASH_SIZE);
        offset += HASH_SIZE;
    }
```

```
    *data_out = buffer;
    *len_out = offset;
    return 0;
}

// —— TODO IMPLEMENTATION ————————————————————————————————————————————————————

// Build a tree from index (minimal valid version for Phase 4)
int tree_from_index(ObjectID *id_out) {

    Index idx;

    if (index_load(&idx) != 0) {
        return -1;
    }

    Tree tree;
    tree.count = 0;

    for (int i = 0; i < idx.count; i++) {

        if (tree.count >= MAX_TREE_ENTRIES) break;

        // Ignore directories (minimal working version)
        if (strchr(idx.entries[i].path, '/')) continue;

        TreeEntry *entry = &tree.entries[tree.count];

        strncpy(entry->name, idx.entries[i].path, sizeof(entry->name));
        entry->name[sizeof(entry->name) - 1] = '\0';

        entry->mode = idx.entries[i].mode;
        entry->hash = idx.entries[i].hash;
```

```
        tree.count++;
    }

    // Must have at least one entry
    if (tree.count == 0) {
        return -1;
    }

    void *data;
    size_t len;

    if (tree_serialize(&tree, &data, &len) != 0) {
        return -1;
    }

    if (object_write(OBJ_TREE, data, len, id_out) != 0) {
        free(data);
        return -1;
    }

    free(data);
    return 0;
}
```